Демо 3. Claude Agent SDK

Что делаем

Устанавливаем `claude-agent-sdk` для Python, запускаем `query()`, потом `ClaudeSDKClient`. Включаем CLAUDE.md через `setting_sources`. Пишем custom MCP tool через декоратор `@tool` и `create_sdk_mcp_server`. Собираем мини-скрипт: автоматический ревью коммитов через SDK.

К концу демо вы умеете встраивать Claude Code в свой Python-код, создавать собственные инструменты с доступом к состоянию приложения и управлять сессиями программно.

Подготовка

1. Python ≥ 3.10. Проверить: `python3 --version`.
2. Установить SDK:

```
pip install claude-agent-sdk
```

3. `ANTHROPIC_API_KEY` в переменных окружения:

```
export ANTHROPIC_API_KEY=sk-ant-...
```

4. Папка с любым проектом и CLAUDE.md.

Шаг 1. Минимальный `query()`

Создайте файл `scripts/sdk-hello.py`:

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for msg in query(
        prompt="Перечисли файлы в корне проекта и опиши, что это за
проект.",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Glob"],
            permission_mode="bypassPermissions",
        ),
    ):
        if hasattr(msg, "result"):
            print(msg.result)

asyncio.run(main())
```

Запустим:

```
python3 scripts/sdk-hello.py
```

Claude прочитает папку и выведет описание. Без `permission_mode` скрипт повис бы на первом запросе.

Шаг 2. Почему CLAUDE.md не виден

Добавим в CLAUDE.md правило:

```markdown
# Test rule

Если спрашивают про проект, начинай ответ со слова «Привет».
```

Запустим скрипт снова. Ответ НЕ начинается с «Привет». Это критический нюанс SDK: по умолчанию CLAUDE.md не загружается.

Шаг 3. Включаем CLAUDE.md через setting_sources

Меняем опции:

```
options=ClaudeAgentOptions(
    allowed_tools=["Read", "Glob"],
    permission_mode="bypassPermissions",
    setting_sources=["project"],
    system_prompt={"type": "preset", "preset": "claude_code"},
)
```

Запустим снова. Теперь ответ начинается с «Привет» — CLAUDE.md подхватился. `preset: "claude_code"` — дополнительно подключает системный промпт настоящего Claude Code.

Шаг 4. Все входящие сообщения

Изменим цикл, чтобы видеть поток сообщений целиком:

```
async for msg in query(prompt="...", options=options):
    print(type(msg).__name__, msg)
```

Увидите: `SystemMessage` (инициализация), `AssistantMessage` (ответы с tool_use), `UserMessage` (результаты tool_use), `ResultMessage` (итоговый ответ с `result`-полем).

Это тот же формат, что `--output-format stream-json` в CLI. SDK даёт типизированные объекты.

Шаг 5. ClaudeSDKClient — persistent-сессия

Создайте `scripts/sdk-session.py`:

```python
import asyncio
from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions

async def main():
    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Glob", "Grep"],
        permission_mode="bypassPermissions",
        setting_sources=["project"],
        system_prompt={"type": "preset", "preset": "claude_code"},
        model="claude-sonnet-4-6",
    )

    async with ClaudeSDKClient(options=options) as client:
        await client.query("Прочитай src/ и назови основные модули.")
        async for msg in client.receive_response():
            if hasattr(msg, "result"):
                print("MODULES:", msg.result)

        await client.query("Для каждого модуля предложи один тест.")
        async for msg in client.receive_response():
            if hasattr(msg, "result"):
                print("TESTS:", msg.result)

asyncio.run(main())
```

Второй `query()` помнит контекст первого — Claude знает, какие модули.

Шаг 6. Custom MCP Tool

Создайте `scripts/sdk-custom-tool.py`:

```python
import asyncio
from claude_agent_sdk import (
    query, tool, create_sdk_mcp_server, ClaudeAgentOptions,
)

# Мок базы данных
USERS_DB = {
    "anna@example.com": {"name": "Anna", "role": "editor"},
    "boris@example.com": {"name": "Boris", "role": "admin"},
}
```

```python
@tool(
    "lookup_user",
    "Найти пользователя по email в базе",
    {"email": str},
)
async def lookup_user(args):
    user = USERS_DB.get(args["email"])
    if not user:
        return {
            "content": [
                {"type": "text", "text": f"Not found: {args['email']}"}
            ]
        }
    return {
        "content": [
            {"type": "text", "text": f"User: {user['name']}, Role:
{user['role']}"}
        ]
    }

users_server = create_sdk_mcp_server(
    name="user-tools",
    version="1.0.0",
    tools=[lookup_user],
)

async def main():
    options = ClaudeAgentOptions(
        mcp_servers={"users": users_server},
        allowed_tools=["mcp__users__lookup_user"],
        permission_mode="bypassPermissions",
    )

    async for msg in query(
        prompt="Найди anna@example.com и опиши её роль.",
        options=options,
    ):
        if hasattr(msg, "result"):
            print(msg.result)

asyncio.run(main())
```

Запустим. Claude вызовет `mcp__users__lookup_user` с аргументом `{"email": "anna@example.com"}`, получит ответ, сформирует человеческий.

Имя инструмента в allowlist: `mcp__<server_name>__<tool_name>`. У нас сервер — `users`, инструмент — `lookup_user`, итого — `mcp__users__lookup_user`.

Шаг 7. Практический пример — ревью коммита

Создайте `scripts/review-last-commit.py`:

```
import asyncio
import subprocess
from pathlib import Path
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    diff = subprocess.run(
        ["git", "diff", "HEAD~1", "HEAD"],
        capture_output=True, text=True, check=True,
    ).stdout

    if not diff.strip():
        print("Нет изменений между HEAD~1 и HEAD")
        return

    prompt = f"""Ревью последнего коммита. Найди:
1. Баги.
2. Проблемы безопасности.
3. Пропущенные тесты.

Формат: markdown с заголовками.

Diff:

{diff}
"""

    options = ClaudeAgentOptions(
        allowed_tools=["Read", "Glob", "Grep"],
        permission_mode="bypassPermissions",
        setting_sources=["project"],
        system_prompt={"type": "preset", "preset": "claude_code"},
        max_turns=10,
    )

    result = ""
    async for msg in query(prompt=prompt, options=options):
        if hasattr(msg, "result"):
            result = msg.result

    Path(".last-commit-review.md").write_text(result)
```

```
    print("Ревью в .last-commit-review.md")

asyncio.run(main())
```

Запустим:

```
python3 scripts/review-last-commit.py
cat .last-commit-review.md
```

Claude читает файлы из диффа, проверяет их состояние, возвращает отчёт.

Шаг 8. Контроль расходов

Все опции:

```
options = ClaudeAgentOptions(
    allowed_tools=["Read", "Glob", "Grep"],
    disallowed_tools=["Bash"],              # перекрывает
bypassPermissions
    permission_mode="acceptEdits",          # вместо bypassPermissions
    model="claude-sonnet-4-6",              # sonnet дешевле opus
    max_turns=10,                           # потолок шагов
    setting_sources=["project"],
    system_prompt={"type": "preset", "preset": "claude_code"},
)
```

Рекомендации:

- `max_turns` — обязательно.
- Sonnet 4.6 для рутины. Opus 4.7 — только для сложных рассуждений.
- `disallowed_tools` для полного запрета — сильнее `allowed_tools`.
- `acceptEdits` вместо `bypassPermissions` — блокирует опасные shell-команды.

Шаг 9. Cron + SDK

Создайте скрипт `scripts/daily-report.py` с тем же паттерном, что выше. Добавьте в crontab:

```
crontab -e
```

Строка для ежедневного запуска в 9:00:

```
0 9 * * * cd /path/to/project && ANTHROPIC_API_KEY=... /usr/bin/python3
scripts/daily-report.py >> logs/daily.log 2>&1
```

Claude каждый день формирует отчёт. Так строятся автопилоты на SDK.

Шаг 10. TypeScript-альтернатива

Для Node.js-проекта установка:

```
npm install @anthropic-ai/claude-agent-sdk
```

Эквивалент `sdk-hello.py`:

```
import { query } from "@anthropic-ai/claude-agent-sdk";

async function main() {
  for await (const message of query({
    prompt: "Перечисли файлы в корне и опиши проект.",
    options: {
      allowedTools: ["Read", "Glob"],
      permissionMode: "bypassPermissions",
      settingSources: ["project"],
      systemPrompt: { type: "preset", preset: "claude_code" },
    },
  })) {
    if ("result" in message) {
      console.log(message.result);
    }
  }
}
```

```
main();
```

CamelCase вместо snake_case. Остальная логика совпадает.

Чеклист

- [ ] Установлен `claude-agent-sdk` через `pip`.
- [ ] `ANTHROPIC_API_KEY` в переменных окружения.
- [ ] Запустили `sdk-hello.py` с `query()` — получили ответ.
- [ ] Проверили, что CLAUDE.md не грузится без `setting_sources`.
- [ ] Добавили `setting_sources=["project"]` + preset `claude_code` — CLAUDE.md подхватился.
- [ ] Запустили `ClaudeSDKClient` с двумя последовательными `query()` — контекст сохранился.
- [ ] Написали `@tool` для мок-базы, собрали через `create_sdk_mcp_server`.
- [ ] Запустили custom tool, Claude вызвал его по имени `mcp__users__lookup_user`.
- [ ] Собрали `review-last-commit.py` — скрипт сохраняет ревью в файл.
- [ ] Добавили `max_turns` и `acceptEdits` вместо `bypassPermissions` для безопасности.